FACULDADE FATEC PE
FACULDADE DE TECNOLOGIA E CIÊNCIAS

Análise de Algoritmos

- Ordenação por Inserção -

Prof. MSc. Marcos

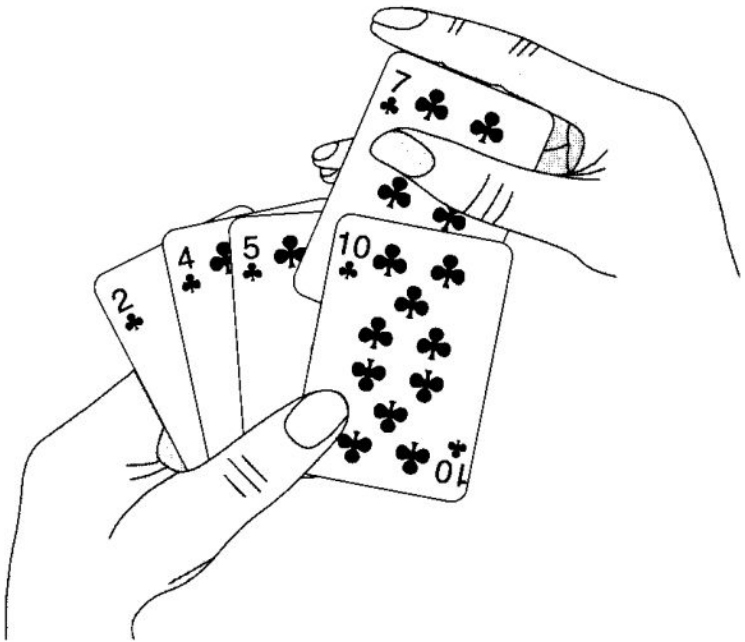
Tipos de Ordenação

- Por Inserção
- Por Intercalação (divisão e conquista)
- Por seleção
- Por troca

FACULDADE FATEC PE
FACULDADE DE TECNOLOGIA E CIÊNCIAS

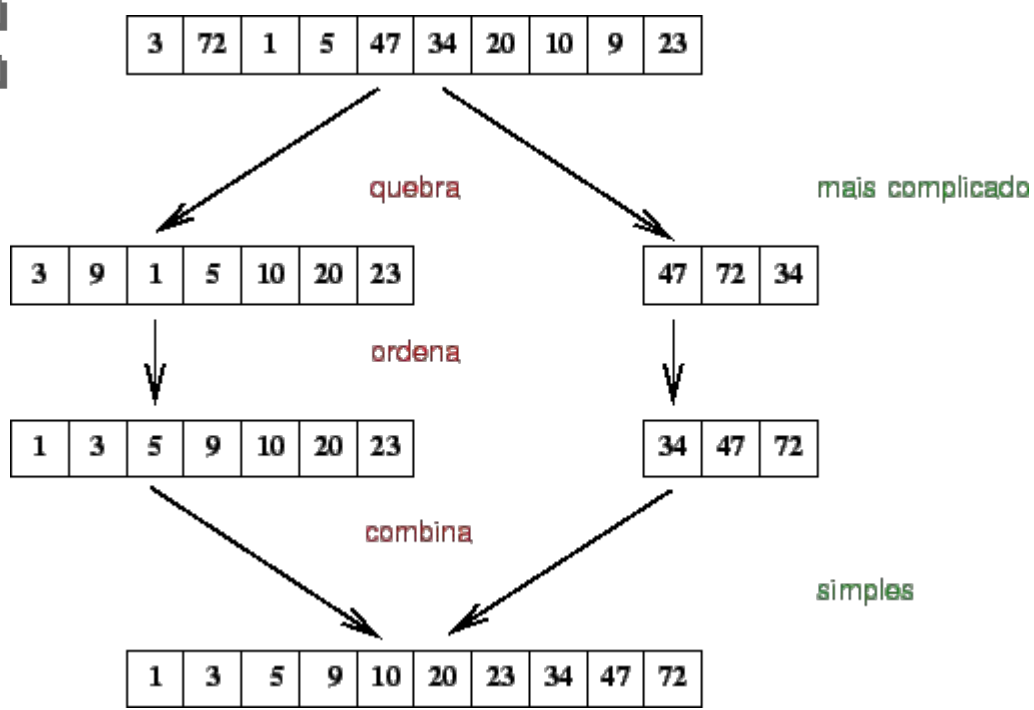
Ordenação por Inserção

-
-



FACULDADE FATEC PE
FACULDADE DE TECNOLOGIA E CIÊNCIAS

Ordenação por Intercalação



Ordenação por Inserção

- **Entrada**: uma sequência de *n* números {A1, A2, A3, ... , An}
- **Saída**: Uma permutação (reordenação) {A1', A2', A3', ... , An' } da sequência de entrada
-
- *__Chave__: número que se pretende ordenar.
-

FACULDADE FATEC PE
FACULDADE DE TECNOLOGIA E CIÊNCIAS

Ordenação por Inserção

```
INSERTION-SORT(A)
for j ← 2 to comprimento[A]
    do chave ← A[j]
        ▷ Inserir A[j] na seqüência ordenada A[1..j − 1].
        i ← j − 1
        while i > 0 e A[i] > chave
            do A[i + 1] ← A[i]
                i ← i − 1
        A[i + 1] ← chave
```

Exemplo:

1	2	3	4	5	6

FACULDADE FATEC PE
FACULDADE DE TECNOLOGIA E CIÊNCIAS

Ordenação por Inserção

Loops Invariantes

- **Inicialização**: ele é verdadeiro antes da primeira iteração.

- **Manutenção**: se for verdadeiro antes de uma iteração se manterá verdadeiro antes da próxima iteração.

- **Término**: quando o loop termina é demonstrado que o algoritmo é correto.

FACULDADE FATEC PE
FACULDADE DE TECNOLOGIA E CIÊNCIAS

Análise de Algoritmo – Insertion Sort

- Tamanho da entrada
-
- Depende o problema a ser estudado
  - Número de itens
  - Número de bits para respresentação
  - Representação por dois números
-
-

FACULDADE FATEC PE
FACULDADE DE TECNOLOGIA E CIÊNCIAS

Análise de Algoritmo – Insertion Sort

- Tempo de Execução
- Total de operações primitivas ou “etapas”
- Avaliará se um algoritmo é mais eficiente que outro

Exemplo: maior elemento

- Considere o algoritmo para encontrar o maior elemento de um vetor de inteiros A[n]; n ≥ 1.

```
#define n 10
int Max(int A[n]) {
    int i, Temp;

    Temp = A[0];
    for (i = 1; i < n; i++)
        if (Temp < A[i])
          Temp = A[i];
    return Temp;
}
```

- Seja *f* uma função de complexidade tal que *f(n)* é o número de comparações entre os elementos de A, se A contiver n elementos.
- **Qual a função f(n)?**

Exemplo: maior elemento

- Considere o algoritmo para encontrar o maior elemento de um vetor de inteiros A[n]; n ≥ 1.

```
#define n 10
int Max(int A[n]) {
   int i, Temp;

   Temp = A[0];
   for (i = 1; i < n; i++)
      if (Temp < A[i])
        Temp = A[i];
    return Temp;
}
```

- Seja *f* uma função de complexidade tal que *f(n)* é o número de comparações entre os elementos de A, se A contiver n elementos.
- Logo ***f(n) = n -1***

Exemplo: maior elemento

- **Teorema**: Qualquer algoritmo para encontrar o maior elemento de um conjunto com n elementos, $n \geq 1$, faz pelo menos $n - 1$ comparações.

Melhor Caso, Pior Caso e Caso Médio

- **Melhor caso**: menor tempo de execução sobre todas as entradas de tamanho n.
- **Pior caso**: maior tempo de execução sobre todas as entradas de tamanho n.
  - Se f é uma função de complexidade baseada na análise de pior caso, o custo de aplicar o algoritmo nunca é maior do que f(n).
- **Caso médio** (ou caso esperado): média dos tempos de execução de todas as entradas de tamanho n.

Ordenação por Inserção

-
-

Demonstração do algoritmo INSERTION-SORT:
http://www.sorting-algorithms.com/insertion-sort

Ordenação por Inserção

- Ineficiente para entradas muito grandes
- Torna-se complexo para tipos variados de entradas (exige mais de uma verificação)
- Algoritmo sempre trará um resultado correto
-
-
- Outro exemplo de algoritmo de ordenação por inserção é o *ShellSort (resolve o problema de ordenação de itens distantes)*